

LAB EXPERIMENTS 11-20

11. Write a C program for possible keys does the Playfair cipher have? Ignore the fact that some keys might produce identical encryption results. Express your answer as an approximate power of 2.

a. Now take into account the fact that some Playfair keys produce the same encryption results. How many effectively unique keys does the Playfair cipher have?

Program:

```
#include <stdio.h>
#include <math.h>

int main()
{
    double log2_25_fact = 83.68;    // log2(25!)
    double log2_2 = 1.0;

    printf("Playfair Cipher Key Analysis\n");
    printf("-----\n");

    printf("1. Total possible keys (ignoring equivalent keys):\n");
    printf("   25! ~ 2^%.2f\n", log2_25_fact);

    printf("\n2. Effectively unique keys (considering equivalent keys):\n");
    printf("   25! / 2 ~ 2^%.2f\n", log2_25_fact - log2_2);

    return 0;
}
```

Output:

```
Playfair Cipher Key Analysis
-----
1. Total possible keys (ignoring equivalent keys):
   25! ~ 2^83.68

2. Effectively unique keys (considering equivalent keys):
   25! / 2 ~ 2^82.68

-----
Process exited after 2.284 seconds with return value 0
Press any key to continue . . . |
```

12. Write a C program to Encrypt the message “meet me at the usual place at ten rather than eight oclock” using the Hill cipher with the key.

9 4

5 7

a. Show your calculations and the result.

b. Show the calculations for the corresponding decryption of the ciphertext to recover the original plaintext.

Program:

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

int main() {
    char text[] = "meet me at the usual place at ten rather than eight oclock";
    char plain[200], cipher[200], decrypt[200];

    int key[2][2] = {{9,4},{5,7}};
    int invKey[2][2] = {{7,22},{21,9}}; // Inverse of key mod 26

    int i, j = 0;

    // Remove spaces and convert to lowercase
    for(i = 0; text[i] != '\0'; i++) {
        if(isalpha(text[i]))
            plain[j++] = tolower(text[i]);
    }
    plain[j] = '\0';

    // Add padding if odd length
    int len = strlen(plain);
    if(len % 2 != 0) {
        plain[len] = 'x';
        plain[len+1] = '\0';
        len++;
    }

    // Encryption
    for(i = 0; i < len; i += 2) {
        int p1 = plain[i] - 'a';
        int p2 = plain[i+1] - 'a';

        cipher[i] = ((key[0][0]*p1 + key[0][1]*p2) % 26) + 'a';
        cipher[i+1] = ((key[1][0]*p1 + key[1][1]*p2) % 26) + 'a';
    }
    cipher[len] = '\0';

    // Decryption
    for(i = 0; i < len; i += 2) {
        int c1 = cipher[i] - 'a';
        int c2 = cipher[i+1] - 'a';

        decrypt[i] = ((invKey[0][0]*c1 + invKey[0][1]*c2) % 26) + 'a';
        decrypt[i+1] = ((invKey[1][0]*c1 + invKey[1][1]*c2) % 26) + 'a';
    }
    decrypt[len] = '\0';

    printf("Plaintext : %s\n", plain);
    printf("Ciphertext : %s\n", cipher);
    printf("Decrypted : %s\n", decrypt);

    return 0;
}
```

Output:

```
Plaintext : meetmeattheusualplaceattenratherthaneightoclockx
Ciphertext : ukixukydromeiwszxwiokunukhxxhroajroanqyebtlkjegad
Decrypted : wqqlwqallppcucafvfaiqallqndalpqdlpanqgypleifeiob
```

```
-----
Process exited after 2.884 seconds with return value 0
Press any key to continue . . . |
```

13. Write a C program for Hill cipher succumbs to a known plaintext attack if sufficient plaintext-ciphertext pairs are provided. It is even easier to solve the Hill cipher if a chosen plaintext attack can be mounted.

Program:

```
// Function to find modular inverse
int modInverse(int a) {
    a %= MOD;
    for (int x = 1; x < MOD; x++) {
        if ((a * x) % MOD == 1)
            return x;
    }
    return -1;
}

int main() {
    // Plaintext pairs: HE LP
    int P[2][2] = {
        {7, 11}, // H=7, L=11
        {4, 15} // E=4, P=15
    };

    // Corresponding Ciphertext pairs: TC EX
    int C[2][2] = {
        {19, 4}, // T=19, E=4
        {2, 23} // C=2, X=23
    };

    // Determinant of P
    int det = (P[0][0] * P[1][1] - P[0][1] * P[1][0]) % MOD;
    if (det < 0)
        det += MOD;

    int invDet = modInverse(det);

    if (invDet == -1) {

        if (det < 0)
            det += MOD;

        int invDet = modInverse(det);

        if (invDet == -1) {
            printf("Plaintext matrix is not invertible.\n");
            return 0;
        }

        // Inverse of P
        int Pinv[2][2];

        Pinv[0][0] = (P[1][1] * invDet) % MOD;
        Pinv[0][1] = ((-P[0][1] + MOD) * invDet) % MOD;
        Pinv[1][0] = ((-P[1][0] + MOD) * invDet) % MOD;
        Pinv[1][1] = (P[0][0] * invDet) % MOD;

        // Recover Key = C x P^-1 (mod 26)
        int K[2][2];

        K[0][0] = (C[0][0]*Pinv[0][0] + C[0][1]*Pinv[1][0]) % MOD;
        K[0][1] = (C[0][0]*Pinv[0][1] + C[0][1]*Pinv[1][1]) % MOD;
        K[1][0] = (C[1][0]*Pinv[0][0] + C[1][1]*Pinv[1][0]) % MOD;
        K[1][1] = (C[1][0]*Pinv[0][1] + C[1][1]*Pinv[1][1]) % MOD;

        printf("Recovered Key Matrix:\n");
        printf("%d %d\n", K[0][0], K[0][1]);
        printf("%d %d\n", K[1][0], K[1][1]);

        return 0;
    }
}
```

Output:

```
Recovered Key Matrix:
1 3
22 1

-----
Process exited after 11.64 seconds with return value 0
Press any key to continue . . . |
```

14. Write a C program for one-time pad version of the Vigenère cipher. In this scheme, the key is a stream of random numbers between 0 and 26. For example, if the key is 3 19 5 . . . , then the first letter of plaintext is encrypted with a shift of 3 letters, the second with a shift of 19 letters, the third with a shift of 5 letters, and so on.

- Encrypt the plaintext send more money with the key stream 9 0 1 7 23 15 21 14 11 11 2 8 9
- Using the ciphertext produced in part (a), find a key so that the cipher text decrypts to the plaintext cash not needed.

Program:

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

int main() {
    char plaintext[] = "sendmoremoney";
    int key[] = {9,0,1,7,23,15,21,14,11,11,2,8,9};

    char ciphertext[100];
    int len = strlen(plaintext);

    printf("Plaintext : %s\n", plaintext);
    printf("Key      : ");

    for(int i = 0; i < len; i++)
        printf("%d ", key[i]);

    printf("\n");

    // Encryption
    for(int i = 0; i < len; i++) {
        int p = plaintext[i] - 'a';
        int c = (p + key[i]) % 26;
        ciphertext[i] = c + 'a';
    }

    ciphertext[len] = '\0';

    printf("Ciphertext: %s\n", ciphertext);

    return 0;
}
```

Output:

```
Plaintext : sendmoremoney
Key       : 9 0 1 7 23 15 21 14 11 11 2 8 9
Ciphertext: beokjdmsxzpmh

-----
Process exited after 2.581 seconds with return value 0
Press any key to continue . . . |
```

15. Write a C program that can perform a letter frequency attack on an additive cipher without human intervention. Your software should produce possible plaintexts in rough order of likelihood. It would be good if your user interface allowed the user to specify “give me the top 10 possible plaintexts.”

Program:

```
#include <ctype.h>

typedef struct {
    int key;
    double score;
    char text[500];
} Result;

// English letter frequencies
double freq[26] = {
    8.167,1.492,2.782,4.253,12.702,2.228,
    2.015,6.094,6.966,0.153,0.772,4.025,
    2.406,6.749,7.507,1.929,0.095,5.987,
    6.327,9.056,2.758,0.978,2.360,0.150,
    1.974,0.074
};

void decrypt(char cipher[], char plain[], int key)
{
    int i;
    for(i = 0; cipher[i] != '\0'; i++)
    {
        if(isalpha(cipher[i]))
        {
            char ch = toupper(cipher[i]);
            plain[i] = ((ch - 'A' - key + 26) % 26) + 'A';
        }
        else
        {
            plain[i] = cipher[i];
        }
    }
}

void sortResults(Result r[])
{
    for(int i = 0; i < 25; i++)
    {
        for(int j = i + 1; j < 26; j++)
        {
            if(r[j].score < r[i].score)
            {
                Result temp = r[i];
                r[i] = r[j];
                r[j] = temp;
            }
        }
    }
}

int main()
{
    char cipher[500];
    Result result[26];

    printf("Enter Ciphertext: ");
    fgets(cipher, sizeof(cipher), stdin);

    for(int key = 0; key < 26; key++)
    {
        result[key].key = key;
        decrypt(cipher, result[key].text, key);
        result[key].score = scoreText(result[key].text);
    }
}
```

Output:

```
Enter Ciphertext: KHOOR ZRUOG
How many top plaintexts to display? 3
Top 3 Possible Plaintexts:

Rank 1
Key   : 6
Score : 1097.53
Text  : EBIIL TLOIA

Rank 2
Key   : 10
Score : 1181.53
Text  : AXEEH PHKEW

Rank 3
Key   : 3
Score : 1285.27
Text  : HELLO WORLD

-----
Process exited after 19.28 seconds with return value 0
Press any key to continue . . . |
```

16. Write a C program that can perform a letter frequency attack on any monoalphabetic substitution cipher without human intervention. Your software should produce possible plaintexts in rough order of likelihood. It would be good if your user interface allowed the user to specify “give me the top 10 possible plaintexts.”

Program:

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

#define MAX 500

char englishFreq[] = "ETAOINSHRDLCLUMNFGYPBVKJXQZ";

typedef struct {
    char letter;
    int count;
} Frequency;

void sortFrequency(Frequency f[])
{
    int i, j;
    Frequency temp;

    for(i = 0; i < 25; i++)
    {
        for(j = i + 1; j < 26; j++)
        {
            if(f[j].count > f[i].count)
            {
                temp = f[i];
                f[i] = f[j];
                f[j] = temp;
            }
        }
    }
}

int main()
{
    sortFrequency(freq);

    printf("\nHow many possible plaintexts? ");
    scanf("%d", &top);

    if(top > 10)
        top = 10;

    for(int k = 0; k < top; k++)
    {
        for(int i = 0; i < 26; i++)
            map[freq[i].letter - 'A'] = englishFreq[(i + k) % 26];

        printf("\nPossible Plaintext %d:\n", k + 1);

        for(int i = 0; cipher[i]; i++)
        {
            if(isalpha(cipher[i]))
            {
                char ch = toupper(cipher[i]);
                printf("%c", map[ch - 'A']);
            }
            else
            {
                printf("%c", cipher[i]);
            }
        }
        printf("\n");
    }
}
```

Output:

```
Enter Ciphertext:
WKL V LV D WHVW PHVVDJH

How many possible plaintexts? 3

Possible Plaintext 1:
ASIE IE O ATEA NTEEHT

Possible Plaintext 2:
OHNT NT I OATO SATTIRA

Possible Plaintext 3:
IRSA SA N IOAI HOAANDO

-----
Process exited after 6.501 seconds with return value 0
Press any key to continue . . . |
```

17. Write a C program for DES algorithm for decryption, the 16 keys (K1, K2, ..., K16) are used in reverse order. Design a key-generation scheme with the appropriate shift schedule for the decryption process.

Program:

```
#include <stdio.h>
#include <string.h>

#define ROUNDS 16

int main()
{
    char keys[ROUNDS][20];

    printf("Enter the 16 encryption keys (K1 to K16):\n");

    for(int i = 0; i < ROUNDS; i++)
    {
        printf("K%d: ", i + 1);
        scanf("%s", keys[i]);
    }

    printf("\nDES Decryption Keys:\n");

    for(int i = ROUNDS - 1; i >= 0; i--)
    {
        printf("K%d = %s\n", ROUNDS - i, keys[i]);
    }

    return 0;
}
```

Output:

```
Enter the 16 encryption keys (K1 to K16):
K1: 1A2B
K2: 2B3C
K3: 3C4D
K4: 4D5E
K5: 5E6F
K6: 6F7G
K7: 7G8H
K8: 8H9I
K9: 9I0J
K10: A1B2
K11: B2C3
K12: C3D4
K13: D4E5
K14: E5F6
K15: F6G7
K16: G7H8

DES Decryption Keys:
K1 = G7H8
K2 = F6G7
K3 = E5F6
K4 = D4E5
K5 = C3D4
K6 = B2C3
K7 = A1B2
K8 = 9I0J
K9 = 8H9I
K10 = 7G8H
K11 = 6F7G
```

18. Write a C program for DES the first 24 bits of each subkey come from the same subset of 28 bits of the initial key and that the second 24 bits of each subkey come from a disjoint subset of 28 bits of the initial key.

Program:

```
#include <stdio.h>
#include <string.h>

int main()
{
    char key56[57];
    char C[29], D[29];
    char subkey[49];

    printf("Enter 56-bit key (0s and 1s):\n");
    scanf("%56s", key56);

    // Split into two 28-bit halves
    strncpy(C, key56, 28);
    C[28] = '\0';

    strncpy(D, key56 + 28, 28);
    D[28] = '\0';

    // Form a 48-bit subkey:
    // First 24 bits from C
    strncpy(subkey, C, 24);

    // Last 24 bits from D
    strncpy(subkey + 24, D, 24);

    subkey[48] = '\0';

    printf("\nLeft Half (28 bits): %s\n", C);
    printf("Right Half (28 bits): %s\n", D);

    printf("\nGenerated 48-bit Subkey:\n");
}
```

Output:

```
Enter 56-bit key (0s and 1s):
11110000110011001010101011110101010101100110011110001111

Left Half (28 bits):  1111000011001100101010101111
Right Half (28 bits): 0101010101100110011110001111

Generated 48-bit Subkey:
111100001100110010101010010101010110011001111000

First 24 bits come from Left Half (C).
Last 24 bits come from Right Half (D).

-----
Process exited after 20.83 seconds with return value 0
Press any key to continue . . . |
```

19. Write a C program for encryption in the cipher block chaining (CBC) mode using an algorithm stronger than DES. 3DES is a good candidate. Both of which follow from the definition of CBC. Which of the two would you choose:

a. For security?

b. For performance?

Program:

```
// Dummy 3DES Encryption (for illustration)
void tripleDES(char block[])
{
    // Replace this with actual 3DES in real applications
    for(int i = 0; i < SIZE; i++)
        block[i] = block[i] + 1;
}

int main()
{
    char plaintext[SIZE + 1];
    char iv[SIZE + 1];
    char temp[SIZE];

    printf("Enter 8-character plaintext: ");
    scanf("%8s", plaintext);

    printf("Enter 8-character IV: ");
    scanf("%8s", iv);

    xorBlock(plaintext, iv, temp);

    tripleDES(temp);

    printf("\nCiphertext (Hex): ");
    for(int i = 0; i < SIZE; i++)
        printf("%02X ", (unsigned char)temp[i]);

    printf("\n");

    return 0;
}
```

Output:

```
Enter 8-character plaintext: priyadar
Enter 8-character IV: studentx

Ciphertext (Hex): 04 07 1D 1E 05 0B 16 0B
```

```
-----
Process exited after 44.39 seconds with return value 0
Press any key to continue . . . |
```

20. Write a C program for ECB mode, if there is an error in a block of the transmitted ciphertext, only the corresponding plaintext block is affected. However, in the CBC mode, this error propagates. For example, an error in the transmitted C1 obviously corrupts P1 and P2.

a. Are any blocks beyond P2 affected?

b. Suppose that there is a bit error in the source version of P1. Through how many ciphertext blocks is this error propagated? What is the effect at the receiver?

Program:

```
int main() {
    int plaintext[BLOCKS] = {10, 20, 30, 40};
    int key = 15;
    int iv = 5;

    int ecb[BLOCKS], cbc[BLOCKS];
    int i;

    // ECB Encryption
    printf("ECB Encryption:\n");
    for(i = 0; i < BLOCKS; i++) {
        ecb[i] = plaintext[i] ^ key;
        printf("P%d = %d -> C%d = %d\n", i+1, plaintext[i], i+1, ecb[i]);
    }

    // CBC Encryption
    printf("\nCBC Encryption:\n");
    cbc[0] = (plaintext[0] ^ iv) ^ key;
    printf("P1 = %d -> C1 = %d\n", plaintext[0], cbc[0]);

    for(i = 1; i < BLOCKS; i++) {
        cbc[i] = (plaintext[i] ^ cbc[i-1]) ^ key;
        printf("P%d = %d -> C%d = %d\n", i+1, plaintext[i], i+1, cbc[i]);
    }

    printf("\nAssume C1 is corrupted during transmission.\n");
    printf("In ECB: Only P1 is affected.\n");
    printf("In CBC: P1 and P2 are affected. P3 onwards are recovered correctly.\n");

    return 0;
}
```

Output:

```
ECB Encryption:
P1 = 10 -> C1 = 5
P2 = 20 -> C2 = 27
P3 = 30 -> C3 = 17
P4 = 40 -> C4 = 39

CBC Encryption:
P1 = 10 -> C1 = 0
P2 = 20 -> C2 = 27
P3 = 30 -> C3 = 10
P4 = 40 -> C4 = 45

Assume C1 is corrupted during transmission.
In ECB: Only P1 is affected.
In CBC: P1 and P2 are affected. P3 onwards are recovered correctly.
```